
KVPrior: ON THE SUPRISING EFFECTIVENESS OF MARGINALIZING KVCACHES OVER PREFIXES

Masataro Asai, Bo Wu, Dan Gutfreund

MIT-IBM Watson AI Lab

{masataro.asai,bo.wu}@ibm.com, dgutfre@us.ibm.com

ABSTRACT

To avoid costly prefills, non-prefix KVCache methods try to reuse individually prefilled KV caches in non-prefix positions. To recover the accuracy degradation, existing methods either perform inference-time recomputation or amortize its cost with model fine-tuning or link-token training, with little theoretical explanation for why they work. We propose KVPrior, a surprisingly simple *recomputation-free, training-free* method to obtain prefix-independent KVCache. Assuming the independence between retrieved documents, it samples a random set of prefixes from the training set, prefill the caches, and averages the context-dependent noise out. This surprisingly simple method nearly recovers the full prefill accuracy and surpassed existing approaches in tasks where the document independence assumptions hold (Biography and Needle-in-the-Haystack). In multi-hop-reasoning datasets where documents depend on each other, we observed weaker yet non-negligible improvements, and combination with existing approaches achieved the state-of-the-art performance.

1 INTRODUCTION

Retrieval-Augmented Generation (RAG) has become a standard approach for augmenting large language models (LLMs) with external documents retrieved from a large corpus. Its major bottleneck is the computational cost of the *prefill* stage, which computes the key-value (KV) cache for the retrieved documents before generating a response. A natural observation is that retrieved documents are frequently reused across many user queries. This has motivated a growing body of work on KV cache reuse, where documents are individually prefilled offline and their KV caches are stored and reused later. While ideally a serving system could simply concatenate the precomputed KV caches of the retrieved documents with no additional recomputation (NO_RECOMPUTE), transformer KV caches are inherently prefix-dependent, thus they cause substantial quality degradation.

Recently proposed approaches recover the lost quality by selective cache recomputation (Yao et al., 2025; Hu et al., 2025; Yan et al., 2026; Wang et al., 2026b), fine-tuning the model to generate more composable caches (Ma et al., 2025), or learning auxiliary link embeddings that bridge documents (Yang et al., 2026; Chen et al., 2026). While these methods substantially improve performance, they incur additional costs during deployment, require costly model training, and/or rely on a mechanism that remains poorly understood. (What are the link tokens actually learning?)

To minimize the runtime and training cost, our goal is to *find an effective offline method that is also training-free*, which guided our solution. According to the conventional wisdom, the source of quality degradation is twofold: (A) the loss of cross-attention between documents, and (B) position-specific artifacts in the cached KV. While many approaches tackle (A) by restoring the cross attention, no existing approach tries to directly tackle (B) by removing the artifact, to our knowledge. Moreover, addressing (A) requires online computation, which goes against our goal of offline method. By focusing on removing the artifacts, we arrived at a substantially simpler offline solution.

We propose KVPrior (Fig. 1): a cache representation that is invariant to the position *and the prefix*. In many RAG applications, retrieved documents mostly (though not necessarily) contribute independently to answering the query, and the specific prefix preceding each document is largely incidental. Inspired by Bayesian statistics, we turn this intuition into a prior assumption. Under the independence assumption as a prior, any contribution from the prefix, including the empty one, is

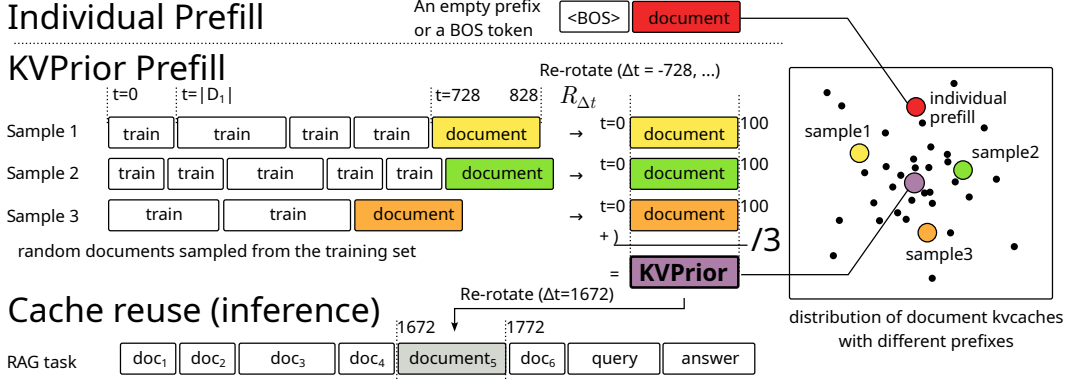


Figure 1: To obtain a position/prefix-independent KV cache, KVPrior estimates the expected cache value by sampling a set of irrelevant documents as prefixes. An individually prefilled KVCache is conditioned by its BOS token prefix or the implicit positional encoding learned by the model and is biased from the mean of KV values over different prefixes.

noise/bias. To remove the bias, we prefill each document multiple times under different randomly sampled prefixes, then compute the average of the resulting KV caches offline. The samples are drawn from the training corpus or from an unrelated dataset that has no chance of data leakage. This smooths out the context-dependent artifacts while preserving document-specific information. The resulting prefix-independent cache can then be reused directly at inference time without any online recomputation or model modification when the prior assumption holds. When the prior assumption does not hold, existing recomputation-based methods can be seen as a posterior update.

Despite its simplicity, KVPrior sometimes recovers the accuracy of full prefill while remaining entirely offline and training-free. *The number of samples needed is as few as 4* to achieve substantial improvements. On retrieval tasks where retrieved documents are largely independent, KVPrior matches or surpasses existing methods. On multi-hop reasoning benchmarks, where interactions between retrieved documents are essential, KVPrior alone provides more modest, yet still substantial improvements, and when combined with cache recomputation methods, it further improves the accuracy. Our contributions are summarized as follows:

- We introduce KVPrior, a simple offline procedure that removes context-dependent noise by averaging KV caches over randomly sampled prefixes.
- The next-token distribution produced by KVPrior approximates the marginal distribution of next-token distribution over arbitrary prefixes of arbitrary lengths.
- KVPrior nearly recovers full-prefill performance on RAG tasks with independent documents.
- KVPrior is complementary to existing non-prefix KV cache methods and can be combined with them to achieve state-of-the-art results on challenging multi-hop reasoning benchmarks.
- Compared to training-based methods, KVPrior generalizes better on out-of-distribution data.

2 PRELIMINARY

Let $1..N$ denote the index sequence $(1, \dots, N)$, and let $\mathbf{c}$ be a sequence of N tokens. Modern language models define an autoregressive distribution over $\mathbf{c}$ by factorizing the joint probability into next-token conditional distributions: $p(\mathbf{c}) = p(\mathbf{c}_{1..j}) \prod_{i=j+1}^N p_{\theta}(\mathbf{c}_i | \mathbf{c}_{1..i-1})$, where θ denotes the model parameters and $\mathbf{c}_{1..j}$ is a prefix of length j , with $1 \leq j < N$.

Transformer (Vaswani et al., 2017) architecture performs autoregressive generation using layers of multi-head self-attention mechanism. Given a hidden state vector $\mathbf{h}[l]$ at layer l , a single self-attention with Rotary Positional Embedding (Su et al., 2024, RoPE) is as follows:

$$\begin{aligned} \mathbf{x} &= \text{Norm}(\mathbf{h}[l]), & \mathbf{K} &= R_t(W^K \mathbf{x}), \\ & & \mathbf{Q} &= R_t(W^Q \mathbf{x}), & \mathbf{O} &= \mathbf{V} \odot \text{SOFTMAX}\left(\frac{\mathbf{Q}^\top \mathbf{K}}{\sqrt{2d}}\right). \\ & & \mathbf{V} &= W^V \mathbf{x}, \end{aligned} \quad (1)$$

$t = 1..N$ is a position and R_t is a RoPE operator. For H self-attentions, the outputs $O_1, \dots, O_H$ are concatenated and fed to an MLP to produce $\mathbf{h}[l+1] = \text{MLP}(O_1; \dots; O_H)$.

Given a $2d$ -dimensional embedding $\mathbf{x}$, RoPE rotates $\mathbf{x}$ to position t as follows. It maintains the following invariance that makes resetting the positions from t to s possible:

$$R(t)\mathbf{x} = \begin{bmatrix} \cos t & -\sin t \\ \sin t & \cos t \end{bmatrix} \begin{bmatrix} \mathbf{x}_{0:d} \\ \mathbf{x}_{d:2d} \end{bmatrix}. \quad \begin{aligned} R(t+s) &= R(t)R(s). \\ \therefore R(s)\mathbf{x} &= R(s-t)(R(t)\mathbf{x}). \end{aligned}$$

This maintains the following invariance that makes resetting the positions from t to s possible:

$$R(t)R(s) = R(t+s), \quad \mathbf{x} = R(t)R(-t)\mathbf{x}, \quad R(s)\mathbf{x} = R(s-t)(R(t)\mathbf{x}).$$

Modern inference engines avoid the $O(N^2)$ complexity in computing $Q^\top K$ by caching the submatrix of K . Given $\mathbf{h}[l]$ and K , the submatrix $K_{1..N-1}$ is identical to the output of the previous iteration, thus the $N-1$ token prefix can be cached. The same applies to V .

$$\mathbf{h}[l] = \begin{bmatrix} \mathbf{h}[l]_{1..N-1} \\ \mathbf{h}[l]_{N-1..N} \end{bmatrix}. \quad K = R(t)W^K \mathbf{h}[l] = \begin{bmatrix} R(t)W^K \mathbf{h}[l]_{1..N-1} \\ R(t)W^K \mathbf{h}[l]_{N-1..N} \end{bmatrix} = \begin{bmatrix} K_{1..N-1} \\ K_{N-1..N} \end{bmatrix}.$$

Computing K and V is often called *prefilling*. We denote the standard prefilling with prefix caching by FULL_RECOMPUTE. Prefix caching reuses the cache only for the common prefix, thus a sequence with a different prefix requires computing the cache from the scratch. This restriction motivates reusing independently prefilled caches at non-prefix positions, known as *position-independent caching*, *non-prefix KV cache reuse*, *individual prefill* and so on. The naive baseline, NO_RECOMPUTE, prefills each document D_k independently to obtain $C_k = (K_k, V_k)$, then rerotates and concatenates them:

$$\tilde{C}^{\text{NR}} = C'_1; \dots; C'_K, \quad C'_k = R_{-\Delta_k}(C_k), \quad \Delta_k = \sum_{i=1}^{k-1} |D_i|.$$

Here, $;$ denotes token-wise concatenation, and $R_{-\Delta_k}$ shifts the keys' RoPE positions backward by Δ_k at every layer, leaving the values unchanged. This aligns positional encoding but cannot recover the cross-document context captured by joint prefilling, potentially degrading answer quality.

Existing methods address this degradation through online recomputation or additional training. *Selective Recomputation* recomputes KV representations for a subset of tokens. Since this requires the retrieved documents D , the recomputation happens during the inference, which causes a random overhead. The number of recomputed tokens controls the runtime-accuracy trade-off; recomputing all tokens recovers FULL_RECOMPUTE while 0% recomputation corresponds to NO_RECOMPUTE. EPIC (Hu et al., 2025) recomputes the tokens at the beginning of each document, while CacheBlend (Yao et al., 2025) first recomputes the first layer for all tokens and then selects the tokens with the largest first layer cache difference. As a baseline, RAND_RECOMPUTE recomputes the caches for randomly selected token positions. *Training-based methods* adapt the model and/or insert auxiliary tokens with learned embeddings. BlockAttention (Ma et al., 2025) fine-tunes the model to accommodate independently prefilled caches. KVLink (Yang et al., 2026) combines fine-tuning with additional *link tokens* between documents. Link tokens are prefilled online and therefore have a runtime penalty, while full-model fine-tuning incurs training and memory costs. KVPacket (Chen et al., 2026) reduces KVLink's training/inference costs by optimizing only the embeddings and prefilling them with the document offline.

3 THE SYMPTOM: CONTEXT-SENSITIVE ARTIFACTS

One potential source of degradation (B) is "attention sink" phenomena (Xiao et al., 2024; Hu et al., 2025; Chen et al., 2026), where the tokens at the beginning of the input tend to gain a large attention mass. Conventional wisdom attributes this to positional embedding since the only difference between the same token at different positions in a sequence is positional embedding.

The issue with this attribution is that there is ample evidence in the existing work that position-specific artifacts remain in the KV cache even without the positional embedding. NoPE (Kazemnejad et al., 2023) showed that transformer models can learn their own positional embedding. DroPE (Gelberg et al., 2026) further supports this claim by showing that it can be removed by calibration while

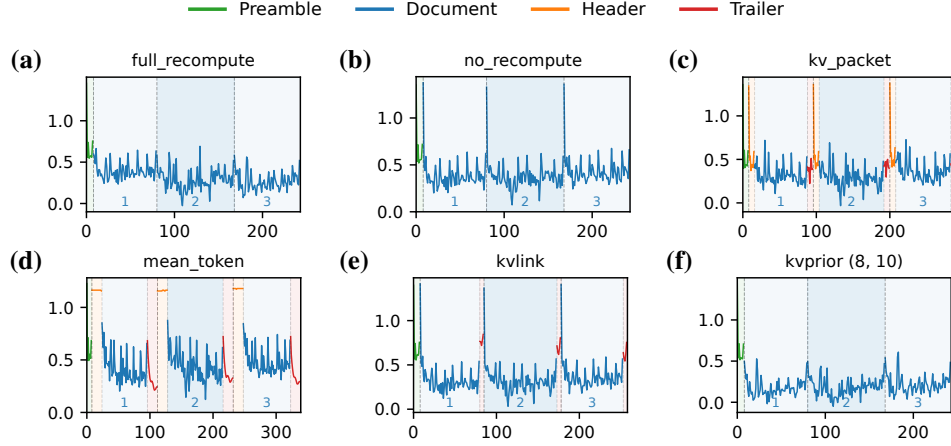


Figure 2: Pre-softmax logits summed over attention heads, layers, and query tokens (y -axis), plotted over token positions in the x -axis (the first 3 documents of the first instance in Biography). **(a)** FULL_RECOMPUTE is a regular prefill. **(b)** NO_RECOMPUTE causes attention spikes at the beginning of every document. **(c)** KVPacket’s header embeddings absorb attention spikes, while minimizing the effect on the semantic meaning. **(d)** Mean-Token is a KVPacket whose embedding is replaced by the mean of the entire vocabulary embedding. **(e)** KVLink does not address the attention spike because its link tokens are placed after each document. **(f)** KVPrior’s attention closely resembles **(a)**.

retaining performance. The implicit position encoding, learned by the model itself and embedded into the representation, cannot be removed by a simple RoPE reset.

Another source is the BOS token at the beginning of inputs in some tokenizers, namely Llama. The effect of such tokens can propagate to later tokens over the layers, similar to the boundary effect in Convolutional Neural Networks (Amari, 1998) due to padding (Kayhan & Gemert, 2020).

To deepen our understanding of such position-specific artifacts, we examine the resulting attention patterns on the Biography (Karev, 2025) benchmark using pre-softmax attention logits from the RAG question tokens to the retrieved document tokens (Fig. 2). Our comparison includes FULL_RECOMPUTE, NO_RECOMPUTE, KVPacket, and KVLink, extending the visualization of Chen et al. to include FULL_RECOMPUTE and KVLink.

Compared with FULL_RECOMPUTE **(a)**, NO_RECOMPUTE **(b)** exhibits large logit spikes at the beginning of each document, despite the positional alignment of each cache C'_k . This observation is consistent with the motivation behind EPIC (Hu et al., 2025), which recomputes the tokens at the beginning of each document. KVPacket **(c)** shows that the learned embeddings absorb the attention spikes. Importantly, KVPacket *does not remove the attention sink*; instead, it learned the embeddings that cause minimal effects even if they receive a large attention mass. To verify this, we include MEAN_TOKEN **(d)**, a KVPacket variant that replaces the learned embedding with the mean of the vocabulary embeddings. It similarly absorbs the spike, thus the specific packet design does not affect the attention sink behavior. Yet it does not improve the downstream RAG performance (see Sec. 5), which supports our observation that KVPacket learns transparent / meaning-free embeddings. Finally, KVLink **(e)** shows large spikes. Unlike KVPacket, its link tokens are positioned after the document and therefore cannot modify the attention at the beginning.

This visualization suggests that neither KVPacket nor KVLink suppresses the attention sinks. At the same time, it also suggests that we can remove the attention spikes without learning the embeddings just by manipulating the prefix context: in FULL_RECOMPUTE **(a)**, the first document exhibits a boundary spike, whereas subsequent documents show fewer spikes and broadly similar logit scales.

4 THE CAUSE: PREFIX BIAS, AND THE SOLUTION: KVPRIOR

The observations above motivate a surprisingly simple, completely offline, training-free method. We propose *KVPrior* (Fig. 1, Alg. 1), which results in the attention score plot (f) which lacks the spikes. As seen in Fig. 2, *non-prefix positions tend not to have spikes*. Therefore, one intuitive way to remove the attention spike is to append a random, meaningless/irrelevant prefix to the target document, prefill the KV cache, remove the prefix, then reset RoPE. Obviously, this leaves some trace of the irrelevant information in the KV cache, which may harm the accuracy downstream. We smooth out this information by sampling ≈ 8 random prefixes and computing the average, leaving only the common elements.

The subfigure on the right in Fig. 1 depicts what is happening in the KV cache space. The standard individual prefill (red), sometimes prefilled with a BOS token prefix depending on the tokenizer, may be biased from the true mean cache values over a variety of prefixes. By sampling many random prefixes (sample1, sample2, sample3), we estimate it via Monte-Carlo sampling.

We interpret this method from a Bayesian point of view. Our prior assumption is that *documents are independent from each other*, and thus independently contribute to the downstream generation. For example, for three documents D_1, D_2, D_3 , we adopt a mean-field assumption $p(D_1, D_2, D_3) = p(D_1)p(D_2)p(D_3)$. One may argue that the naive individual prefill in NO_RECOMPUTE computes exactly $p(D_k)$, but this is not the case. In the individual prefill, every input is prefixed by BOS or an empty prefix, thus the distribution obtained by feeding each document to the model is actually $p(\mathbf{c}_{1..N} = D_k | \mathbf{c}_0 = \text{BOS or ""})$ where $N = |D_k|$, rather than $p(\mathbf{c}_{1+T..N+T} = D_k)$ for an arbitrary prefix $\mathbf{c}_{1..T}$ of an arbitrary length T . To address this, Alg. 1 performs the following marginalization:

$$p(D_k) = \mathbb{E}_{p(m)} \mathbb{E}_{p(D'_1 \dots D'_m | m)} [p(D_k | D'_1 \dots D'_m)]$$

where $p(D'_1 \dots D'_m | m)$ uniformly and independently draws m documents from a set of documents and concatenates them. In a RAG benchmark, the set is the union of all documents in all tasks in the training set of the benchmark. As our mean-field assumption is merely a prior, it is *possible and permissible* to be wrong; a prior is meant to be updated via observation into a posterior. This also frames the role of cache recomputation as a sort of posterior update to the KVPrior.

4.1 THEORETICAL JUSTIFICATION

There is still a theoretical gap between obtaining the prefix/position-independent prior distribution $p(D_k)$ and “averaging the KV Cache”. We analyze the gap in this section. Let $\mathbf{z} = \mathbf{c}_{1..N-1}$. Let $C(\mathbf{z})$ be the KV cache of $\mathbf{z}$, which is a deterministic function of $\mathbf{z}$. In the transformer architecture, $p(\mathbf{c}_N | \mathbf{z})$ is also a deterministic function of $C(\mathbf{z})$. Let $p(\mathbf{c}_N | \mathbf{z}) = f(C(\mathbf{z}))$. Recall that the distribution of the N -th token is modeled as $p(\mathbf{c}_N) = \mathbb{E}_{p(\mathbf{z})} [p(\mathbf{c}_N | \mathbf{z})]$, therefore $p(\mathbf{c}_N) = \mathbb{E}_{p(\mathbf{z})} [f(C(\mathbf{z}))]$. Our goal is to approximate this with $\tilde{C} = \frac{1}{M} \sum_{i=1}^M C_i$ (Alg. 1, line 9). Let $\mu = \mathbb{E}_{p(\mathbf{z})} [C(\mathbf{z})]$ and $C(\mathbf{z}) = \mu + \epsilon$. Then the Taylor expansion of f around μ is approximated as follows. Note that $\mathbb{E}_{p(\mathbf{z})} [\epsilon] = 0$.

$$f(C(\mathbf{z})) = f(\mu + \epsilon) = f(\mu) + J_f(\mu)\epsilon + \frac{1}{2}\epsilon^\top H_f(\mu)\epsilon + O(\|\epsilon\|^3). \quad (2)$$

$$p(\mathbf{c}_N) = \mathbb{E}_{p(\mathbf{z})} [f(C(\mathbf{z}))] = f(\mu) + J_f(\mu) \cdot 0 + \frac{1}{2} \mathbb{E}_{p(\mathbf{z})} [\epsilon^\top H_f(\mu)\epsilon] + O(\mathbb{E}_{p(\mathbf{z})} [\|\epsilon\|^3]) \quad (3)$$

$$\approx f(\mu) = f(\mathbb{E}_{p(\mathbf{z})} [C(\mathbf{z})]) \quad (4)$$

$$\approx f(\tilde{C}). \quad (\text{Monte-Carlo estimate.}) \quad (5)$$

The key step in this approximation is Eq. 4 which assumes that the hessian of f around μ in the direction of cache variation ϵ is small. Intuitively, this can be interpreted as follows: *since we randomly*

sample prefix documents $\mathbf{z}$ that are irrelevant to $\mathbf{c}_N$ in the target document, ϵ is orthogonal to H_f and does not affect f and $p(\mathbf{c}_N)$. We empirically validate this claim in Sec. 5.

5 EMPIRICAL EVALUATION

We evaluate each KV cache reuse method on RAG tasks. A RAG task is a set of 3-tuples $\Pi = (Q, A, D)$ where Q is the question, A is the ground-truth answer, and $D = (D_1, \dots, D_K)$ is a sequence of K documents some of which contain information relevant to Q and A . We assume D is given and focus on generating A , while retrieving D from a larger corpus is outside the scope of this paper. Using a language model $p_\theta(A|Q, D)$, the goal of the task is to sample an answer $\tilde{A}$ that is as close to A as possible. The performance is measured by two metrics: the *F1 metric*, which is the token-level Jaccard distance between A and $\tilde{A}$, and *TTFT* (Time to First Token), which is the sum of the runtime for any additional processing for KV caches and the runtime for prefilling the query Q . TTFT does not include the runtime for prefilling the system prompt. Due to the different runtime-accuracy trade-offs, proper evaluation requires comparing its Pareto front.

In addition, LLMs are known to exhibit positional biases (Liu et al., 2024; Glater & Santos, 2023). To remove the bias, we compute the scores over 6 random permutations of the original document ordering in the dataset, including the single original ordering, and compute the average score. Further implementation details are available in Sec. A1.

In all experiments, we used three models: Llama-3.1-8B-Instruct (Grattafiori et al., 2024), Qwen3-4B-Instruct-2507 (Yang et al., 2025), Granite-4.1-8b (Shah, 2026). We used five RAG datasets: Biography (Karev, 2025), RULER / Needle-in-a-Haystack (Niah) (Jegou, 2024), HotpotQA (Yang et al., 2018), Musique (Trivedi et al., 2022), TwoWikiQA (Ho et al., 2020). In Biography and RULER, questions can be answered without combining information across documents, so the document independence (mean-field) assumption holds. The other three datasets require multi-hop reasoning.

We evaluated KVPrior with $(N, M) = (4, 5)$ and $(8, 10)$, then compared/combined them with CacheBlend, EPIC, and RAND_RECOMPUTE that recomputes 10%, 20%, or 40% of document tokens. We also trained KVLink with $L = 0, 1, 5$ link tokens ($L = 0$ corresponds to BlockAttention (Ma et al., 2025)), KVPacket with 8/8 header/trailer tokens (the default value in the official repository), and MEAN_TOKEN with 8/8, 16/16, 32/32, 64/64 header/trailer tokens. The recomputation budget in EPIC was changed from the original, which sets a fixed number of tokens as a hyperparameter, because it does not allow equal comparisons with CacheBlend.

5.1 RUNTIME-ACCURACY PARETO FRONT

KVPrior vs. Recomputation, KVPacket, KVLink Fig. 3 compares the accuracy-runtime trade-offs of various methods. Since KVPacket, KVPrior, and KVLink with $L = 0$ are fully offline methods, they show little runtime degradation against the NO_RECOMPUTE baseline.

We first look at Biography and Niah benchmarks that contain no multi-hop reasoning and thus no dependency between retrieved documents, which are ideal targets of KVPrior. On Biography, KVPrior substantially improves over NO_RECOMPUTE and achieves the best Pareto front among all methods, approaching FULL_RECOMPUTE accuracy. On Niah, KVPrior approaches FULL_RECOMPUTE in Granite, with slightly wider gaps in other models. Its Pareto front slightly lags behind KVLink in Llama/Qwen, and is comparable in Granite.

On the multi-hop benchmarks (HotpotQA, Musique, and 2WikiQA) it was not designed for, KVPrior improves over NO_RECOMPUTE yet the improvement is more modest. This is expected, as KVPrior is an offline method and does not address cross-attention. Compared to other offline methods, KVPrior is comparable with KVPacket overall, and only slightly lags behind KVLink ($L = 0$), both of which require training with substantially more training samples.

The difference of performance between two hyperparameter configurations $(N, M) = (4, 5)$ and $(8, 10)$ was not significant. Both configurations improved over NO_RECOMPUTE, indicating that a small number of sampled prefixes are already enough.

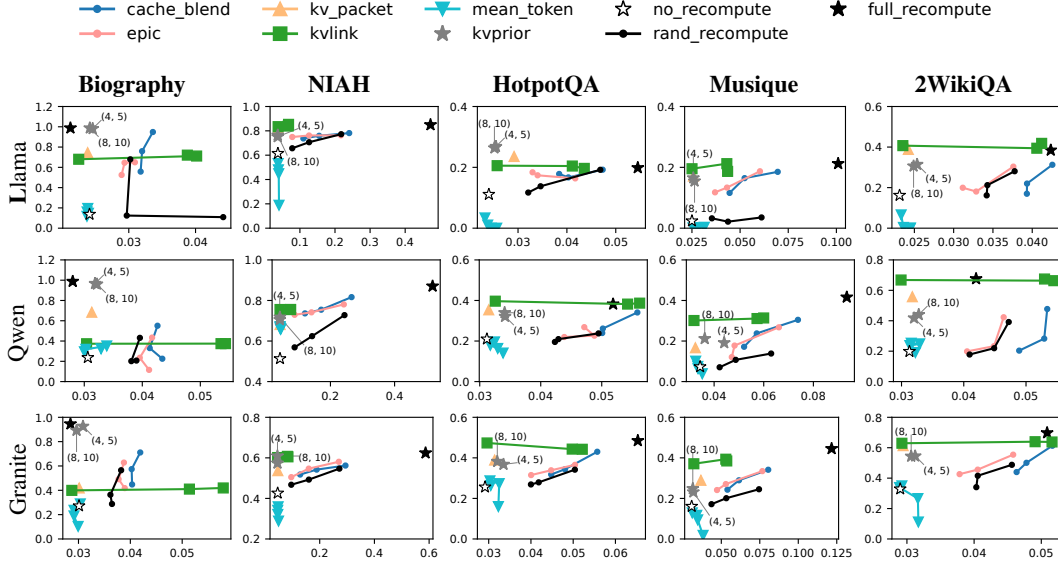


Figure 3: F1 scores (y -axis) vs. TTFT (x -axis). Despite its extreme simplicity, its training-free nature, and its use of merely 4 or 8 training samples vs. the entire dataset, KVPrior is generally on par with (and sometimes outperforming) KVPacket.

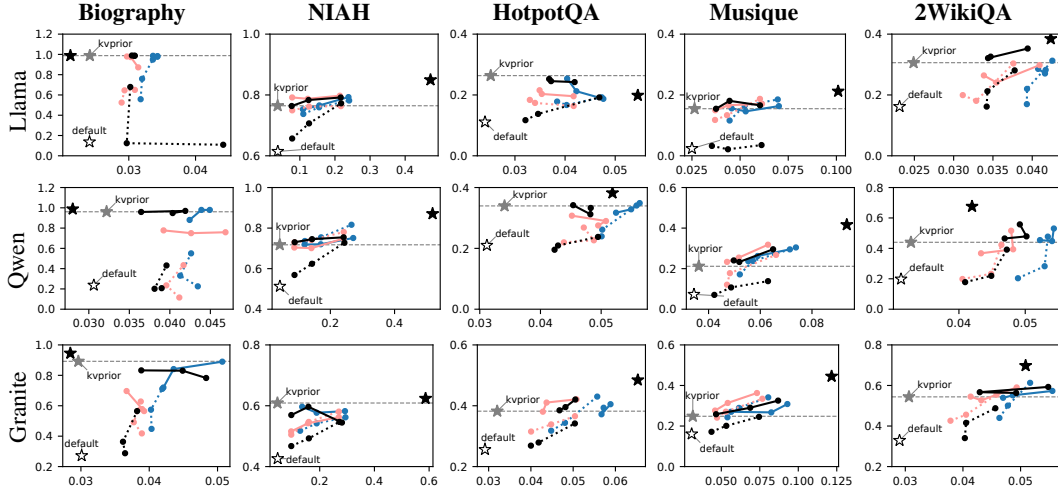


Figure 4: (Legends follow Fig. 3.) Applying recombination methods to KVPrior caches (solid lines) often improves the accuracy over using KVPrior alone (gray $\star$). Exceptions are (1) when FULL_RECOMPUTE performed worse than KVPrior in HotpotQA/Llama (thus more recombination makes it worse), and (2) when token selection in CacheBlend / EPIC is uninformative and RAND_RECOMPUTE performs better (e.g., HotpotQA/TwoWikiQA). Moreover, KVPrior+recombination tends to outperform the recombination method alone (dotted lines).

Combining KVPrior with Recombination Methods To improve the accuracy on multi-hop benchmarks, we combine KVPrior with recombination methods. Specifically, we apply recombination methods to $\tilde{C}$ to select some tokens and then recompute sub-tensors of $\tilde{C}$ in those positions. Fig. 4 shows the performance of such settings with $(N, M) = (8, 10)$. KVPrior+Recombination generally improves the accuracy over KVPrior alone as well as recombination methods alone, demonstrating that such a combination is promising.

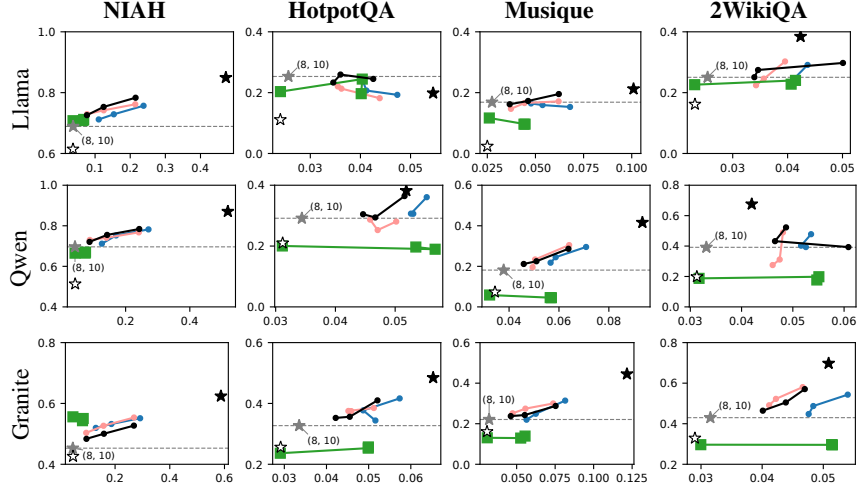


Figure 5: (Legends follow Fig. 3.) OOD generalization comparison. KVPrior caches are prefilled with prefixes sampled from Biography and combined with recomputation methods, and KVLink is trained on Biography. KVPrior improves the accuracy except in Granite/Niah while KVLink struggles to generalize in multi-hop datasets.

Out-of-Distribution Generalization We also evaluate whether KVPrior’s gain extends to out-of-distribution documents. Fig. 5 shows the result of KVPrior and its combination with recomputation methods that use random Biography documents as prefixes when prefilling documents from Niah, HotpotQA, Musique, and TwoWikiQA. KVPrior still improves over NO_RECOMPUTE, and the combination with recomputation methods also improves the accuracy similar to Fig. 4.

To put its OOD generalization into perspective, we also trained KVLink on the Biography dataset and evaluated it on Niah, HotpotQA, Musique, and TwoWikiQA. Unlike KVPrior, KVLink significantly degraded its accuracy compared to in-distribution evaluation, indicating that it can no longer handle OOD documents because of the model training. With Qwen and Granite, KVLink barely improves the performance in multi-hop datasets even though it improved Biography.

5.2 MEASURING THE HESSIAN CONTRIBUTION

To validate the theoretical claim in Sec. 4.1, we measure the *total variation* (TV) between a Monte-Carlo estimate of $A = \mathbb{E}_{p(\mathbf{z})}[f(C(\mathbf{z}))]$ and the distribution $B = f(\tilde{C})$ from KVPrior. TV between two discrete distributions $\mathbf{p}, \mathbf{q}$ is defined as $TV(\mathbf{p} \parallel \mathbf{q}) = \frac{1}{2} \sum_{i \in V} |\mathbf{p}_i - \mathbf{q}_i|$, where V is the vocabulary. It bounds the probability difference $|\mathbf{p}_i - \mathbf{q}_i|$ from above for any token i . TV also corresponds to the sum of absolute values of rows in $\frac{1}{2} \mathbb{E}_{p(\mathbf{z})}[\epsilon^\top H_f(\mu) \epsilon]$ up to the cubic term. We measured them across 3 models, 5 benchmarks, and with in-distribution prefixes (sampled from the training set of the same benchmark) and out-of-distribution prefixes (sampled from Biography).

Table 1 shows the results on four models and five benchmarks. The TV value tends to be below 0.1 except in Granite, where the value could be as high as 0.12. As expected, the TV reduced in the OOD prefixes because the documents in Biography have much less chance of being relevant to the target documents than the training set documents in the same benchmark do.

To put these values into perspective, we also measured the TV values from A to A_{FP8} and A_{INT8} , which are A evaluated under a different quantization (FP8, INT8). They give the relative scale of impact due to the similar error caused by quantization. The result suggests that $TV(A \parallel B)$ tends to be in a scale similar to $TV(A \parallel A_{\text{FP8}})$ and $TV(A \parallel A_{\text{INT8}})$. Granite 8B with INT8 is missing because it does not have the official INT8 quantized weights.

Table 1: Total variation $\text{TV}(A \parallel X)$ of distribution X from the marginal $A = \mathbb{E}_{p(\mathbf{z})}[f(C(\mathbf{z}))]$ with $(N, M) = (8, 10)$. X is either $B = f(\tilde{C})$, i.e., the distribution from KVPrior, or $A_{\text{FP8}}, A_{\text{INT8}}$, i.e., the same marginal under quantizations (FP8, INT8). The bottom half shows when prefixes are sampled from the Biography dataset.

$X =$	Biography			Niah			HotpotQA			Musique			TwoWikiQA		
	B	A_{FP8}	A_{INT8}	B	A_{FP8}	A_{INT8}	B	A_{FP8}	A_{INT8}	B	A_{FP8}	A_{INT8}	B	A_{FP8}	A_{INT8}
TV / In-distribution samples															
Llama	0.11	0.077	0.05	0.15	0.058	0.047	0.057	0.11	0.064	0.064	0.11	0.063	0.086	0.1	0.066
Qwen3	0.11	0.038	0.056	0.13	0.029	0.042	0.055	0.04	0.057	0.048	0.036	0.052	0.079	0.038	0.058
Granite	0.12	0.058	–	0.13	0.042	–	0.071	0.083	–	0.087	0.095	–	0.12	0.098	–
TV / Out-of-distribution samples (Biography)															
Llama	–	–	–	0.0036	0.025	0.012	0.014	0.058	0.021	0.011	0.058	0.02	0.015	0.047	0.02
Qwen3	–	–	–	0.01	0.0058	0.012	0.025	0.011	0.025	0.017	0.0084	0.021	0.026	0.0088	0.03
Granite	–	–	–	0.0043	0.014	–	0.022	0.033	–	0.031	0.041	–	0.041	0.041	–

6 RELATED WORK

As cache reuse is an emerging field, there are many recently-proposed methods which we failed to mention. These include QCFuse (Yan et al., 2026), ProphetKV (Wang et al., 2026b), FusionRAG (Wang et al., 2026a), SAM-KV (Cao et al., 2026).

Position/prefix-independent KVCache has an important application in LLM-based programming frameworks (mel, 2025; Zheng et al., 2024; lan, 2022) for running algorithmic operations over natural language (Lall et al., 2026). Programs that invoke LLMs as a sub-routine, often called *LLM pipelines*, frequently reuse a literal or an intermediate string returned by a model as a substring at a different position of the input prompt to another LLM call.

The mechanism of suppressing the noise from the random prefixes may resemble convolution with a Gaussian kernel in Gaussian smoothing (Witkin, 1987). Deriving a stronger theoretical connection than Eq. 4 from a desirable property of the model (e.g., convexity in Softmax (Gao & Pavel, 2017)) is an interesting avenue for future work.

7 CONCLUSION

We proposed KVPrior, a position-and-prefix-independent KVCache prefilling method that works on a variety of tasks. Inspired by Bayesian statistics, it uses the document independence as a prior assumption, and creates a KV cache that tends to work with a variety of positions and prefixes. We believe ours is the first characterization of position-independent KV Cache from a Bayesian point of view, which opens a variety of avenues for future work. For example, while our method mitigates the bias by averaging, it does not fully represent the prior distribution of the KVCache as, e.g., a Gaussian. We do not propose any specialized mechanism for updating the prior into a posterior either. For example, an interesting avenue of future work includes a custom LoRA adapter that is tailored toward computing the differential update to the KV cache.

AI USE STATEMENT

In this work, we used generative AI tools for

- formulate mathematical claims
- implement methods

We have not used generative AI tools for

- help develop theoretical models or conceptual frameworks
- provide critical ingredients for proving mathematical claims
- assist in the writing of proofs
- propose or refine hypotheses
- design or provide feedback on research methodology or experiments
- assist with translation

- support qualitative and thematic data analysis
- interpret results.

and

- Generate synthetic data sets
- clean and reformat dataset

are not applicable to this work. Additionally, we used generative AI tools for

- create or modify scientific figures or images
- create or edit software code
- draft parts of a research paper
- sourcing/searching for information
- edit a research paper to improve readability
- identify relevant literature

We have reviewed all AI-assisted work. LLM-generated code was mainly an incremental modification to an existing codebase, and each modification is verified and tested for correctness by the main author. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REPRODUCIBILITY STATEMENT

We included the source code as supplements.

REFERENCES

- Langchain, 2022. URL <https://www.langchain.com/>.
- Mellea, 2025. URL <https://mellea.ai/>.
- Shun-Ichi Amari. Natural Gradient Works Efficiently in Learning. *Neural Computation*, 10(2):251–276, 1998.
- Ziyi Cao, Qingyi Si, Jingbin Zhang, and Bingquan Liu. Sparse Attention across Multiple-Context KV Cache. In *Proc. of AAAI Conference on Artificial Intelligence*, volume 40, pp. 30165–30173, 2026.
- Chuangtao Chen, Grace Li Zhang, Xunzhao Yin, Cheng Zhuo, Bing Li, and Ulf Schlichtmann. KV Packet: Recomputation-Free Context-Independent KV Caching for LLMs. *arXiv preprint arXiv:2604.13226*, 2026.
- Bolin Gao and Lacra Pavel. On the Properties of the Softmax Function with Application in Game Theory and Reinforcement Learning. *arXiv preprint arXiv:1704.00805*, 2017.
- Yoav Gelberg, Koshi Eguchi, Takuya Akiba, and Edoardo Cetin. Extending the Context of Pretrained LLMs by Dropping Their Positional Embedding. In *Proc. of the International Conference on Learning Representations (ICLR)*, volume 2026, pp. 60170–60197, 2026.
- Rafael Glater and Rodrygo LT Santos. On Answer Position Bias in Transformers for Question Answering. pp. 2215–2219, 2023.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*, 2024.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a Multi-Hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proc. of the International Conference on Computational Linguistics (COLING)*, pp. 6609–6625, 2020.
- Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *Proc. of the International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

-
- Junhao Hu, Wenrui Huang, Weidong Wang, Haoyi Wang, Tiancheng Hu, Zhang Qin, Hao Feng, Xusheng Chen, Yizhou Shan, and Tao Xie. EPIC: Efficient position-independent caching for serving large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proc. of the International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pp. 24391–24402. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/hu25j.html>.
- Simon Jegou. `simonjegou/ruler. huggingface dataset`, 2024.
- Alex Karev. `alex-karev/biographies. huggingface dataset`, 2025.
- Osman Semih Kayhan and Jan C van Gemert. On Translation Invariance in CNNs: Convolutional Layers can Exploit Absolute Spatial Location. In *Proc. of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14274–14285, 2020.
- Amirhossein Kazemnejad, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Payel Das, and Siva Reddy. The Impact of Positional Encoding on Length Generalization in Transformers. *Proc. of the Advances in Neural Information Processing Systems (Neurips)*, 36:24892–24928, 2023.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with Pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pp. 611–626, 2023.
- Supriya Lall, Christian Farrell, Hari Pathanjaly, Marko Pavic, Sarvesh Chezhian, and Masataro Asai. Verbalized Algorithms: Classical Algorithms are All You Need (Mostly), 2026. URL <https://arxiv.org/abs/2509.08150>.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the Middle: How Language Models use Long Contexts. *Proc. of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 12:157–173, 2024.
- Dongyang Ma, Yan Wang, and Tian Lan. Block-Attention for Efficient Prefilling. In *Proc. of the International Conference on Learning Representations (ICLR)*, volume 2025, pp. 63774–63788, 2025.
- Yousaf Shah. Granite 4.1 LLMs: How They’re Built, 2026. URL <https://huggingface.co/blog/ibm-granite/granite-4-1>.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing*, 568:127063, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop Questions via Single-Hop Question Composition. *Proc. of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 10:539–554, 2022.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All You Need. In *Proc. of the Advances in Neural Information Processing Systems (Neurips)*, pp. 5998–6008, 2017.
- Jiahao Wang, Weiyu Xie, Mingxing Zhang, Boxin Zhang, Jianwei Dong, Yuening Zhu, Chen Lin, Jingqi Tang, Yaochen Han, Zhiyuan Ai, et al. From Prefix Cache to Fusion Rag Cache: Accelerating LLM Inference in Retrieval-Augmented Generation. *Proc. of the International Conference on Management of Data (SIGMOD)*, 4(1):1–28, 2026a.
- Shihao Wang, Jiahao Chen, Yanqi Pan, Hao Huang, Yichen Hao, Xiangyu Zou, Wen Xia, Chongyang Qiu, Wentao Zhang, and PengFei Wang. ProphetKV: User-Query-Driven Selective Recomputation for Efficient KV Cache Reuse in Retrieval-Augmented Generation. In *Proc. of the International Conference on Machine Learning (ICML)*, 2026b. URL <https://openreview.net/forum?id=SwGwmKIgUb>.
- Andrew P Witkin. Scale-space filtering. In *Readings in computer vision*, pp. 329–332. Elsevier, 1987.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient Streaming Language Models with Attention Sinks. In *Proc. of the International Conference on Learning Representations (ICLR)*, volume 2024, pp. 21875–21895, 2024.

-
- Jianxin Yan, Wangze Ni, Zhenxin Li, Jiabao Jin, Zhitao Shen, Haoyang Li, Jia Zhu, Peng Cheng, Xuemin Lin, Lei Chen, et al. QCFuse: Query-Aware Cache Fusion via Compressed View for Efficient RAG Serving. *arXiv preprint arXiv:2606.05875*, 2026.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVlink: Accelerating Large Language Models via Efficient KV Cache Reuse. *Proc. of the Advances in Neural Information Processing Systems (Neurips)*, 38:133797–133824, 2026.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A Dataset for Diverse, Explainable Multi-Hop Question Answering. pp. 2369–2380, 2018.
- Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast Large Language Model Serving for Rag with Cached Knowledge Fusion. In *Proc. of the European Conference on Computer Systems*, pp. 94–109, 2025.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. SGLANG: Efficient Execution of Structured Language Model Programs. *Proc. of the Advances in Neural Information Processing Systems (Neurips)*, 37:62557–62583, 2024.

APPENDIX

A1 IMPLEMENTATION DETAILS

Our KVPrior code is based on the public repository of KVPacket, which uses the bare torch/Huggingface engine as a backend. Integration of KVPrior with State-of-the-Art inference engines such as vllm (Kwon et al., 2023) and SGLang (Zheng et al., 2024) to optimize the throughput is future work but is clearly outside the scope of this paper.

A1.1 KVPACKET TRAINING DETAILS

Our implementation of KVPacket is identical to the public codebase published by the author of KVPacket (Chen et al., 2026). Compared to the public codebase, we slightly increased the number of data points used in the training, then held out 5% of the dataset for plotting the validation loss. We used the same optimization settings for every dataset: 30 epochs, batch size 64, AdamW at learning rate 5×10^{-4} with linear decay to zero.

Benchmark	Dataset size
Biography	$256 = 64 \times [2, 3, 5, 7]$ docs
Niah	128
HotpotQA	512
Musique	512
2wikiQA	512

Table A2: KVPacket training datasets. 5% of them are held-out as a validation set.

A1.2 KVLINK TRAINING DETAILS

Our KVLINK implementation uses a Lora adapter (Hu et al., 2022) with the default PEFT configuration ($r = 8, \alpha = 8$) instead of full-model fine-tuning, and we use the KL divergence distillation loss from KVPacket instead of the Cross-Entropy supervised loss for RAG because the latter conflates the improvements from RAG-specific task adaptation and those from cache reuse adaptation. Lora enabled single-GPU training compared to 8 GPUs in (Yang et al., 2026), yet the whole pipeline still consumed up to 6 hours with NVidia H100 GPUs. Table A3 shows the trainign data statistics.

We cache the teacher logits on the main memory and reuse it over the epochs. To save the memory usage, we discard logits for tokens in the bottom 1% probability mass and flatten it out. This simple procedure achieved almost two orders of magnitude of memory saving, with less than 1% difference in the KL divergence loss.

Training runs for 2 epochs with batch size 16 with AdamW at learning rate starting from 5×10^{-5} with linear decay. Because the adapter has to learn to encode documents and not only to bridge them, the dataset used are an order of magnitude larger than KVPacket’s training data (Table A3). We increased the dataset size until the validation loss no longer shows the sign of overfitting.

Configuration	Data configurations (entries $\times$ docs/entry)
Biography	$10240 = 2560 \times [2, 3, 5, 7]$ docs
Niah	1280
HotpotQA	10240
Musique	10240
2wikiQA	10240

Table A3: KVLINK training datasets. 5% of them are held-out as a validation set.

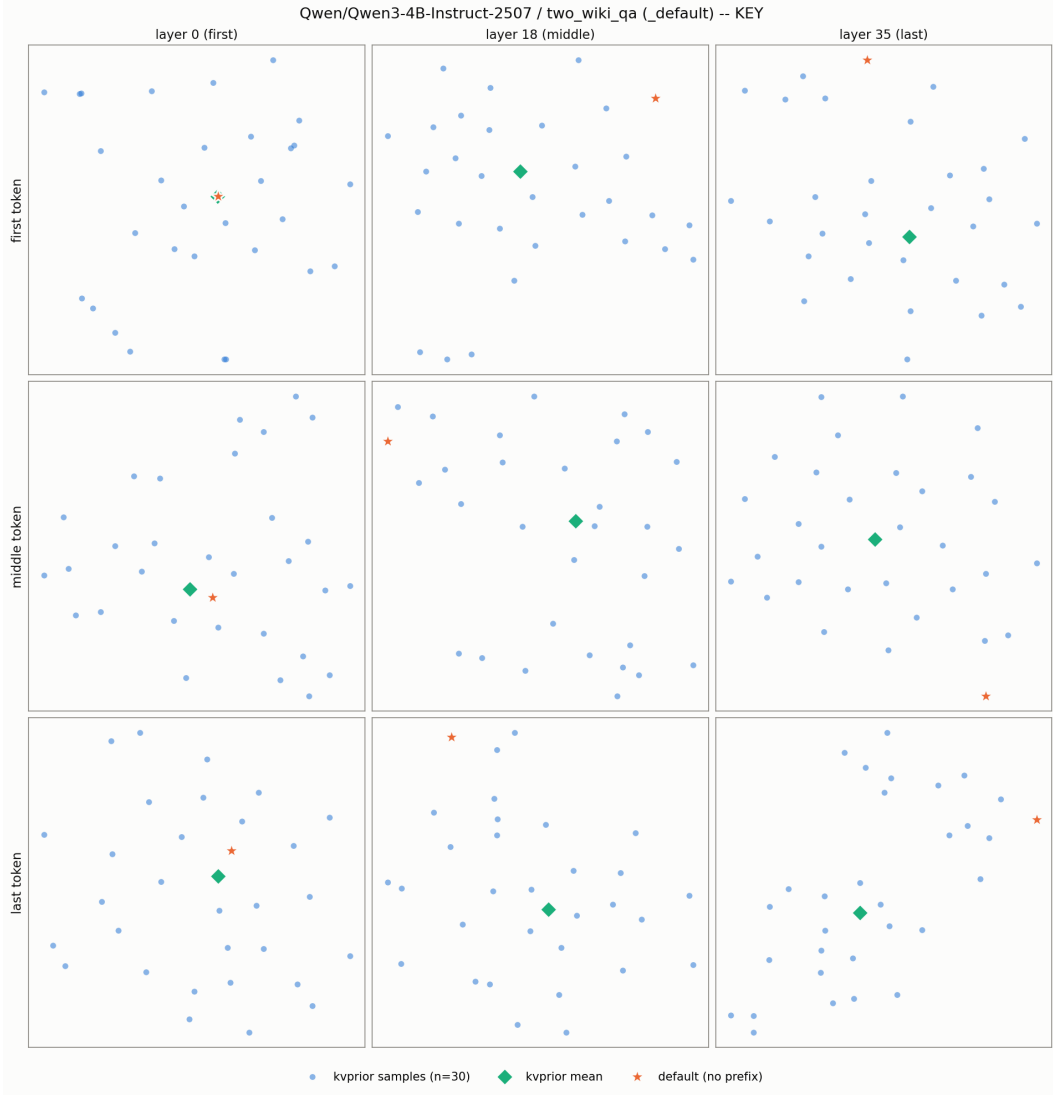


Figure A6: T-SNE visualization of K vector produced by individual prefill, KVPrior, and samples of KVPrior, at various token positions and layers.

A2 VISUALIZING THE KVCACHE DISTRIBUTION

To visualize the space of KV cache, we selected the first document in the first task of every benchmark, then performed an individual prefill and the KVPrior prefill.

Let T be the document length (in tokens) and L is the layer depth. We visualized the ℓ -th layer of the key vector of t -th token via t-SNE, where $(t, \ell) \in \{0, \lfloor T/2 \rfloor, T\} \times \{0, \lfloor L/2 \rfloor, L\}$.

Fig. A6 shows an example from Qwen in Biography. This indicates that the KVCache of individual prefill could be far away from the mean of the cache distribution conditioned by random prefixes.